Ashkorix Local GGUF Runner — Basic Phased Implementation Plan

1. Product Goal

Ashkorix is a local-only desktop application for running `.gguf` language models, importing common document types, searching across user-owned knowledge, and producing cited answers without cloud services, telemetry, or external API dependencies.

The first milestone should focus on one simple promise:

> Import files, ask questions, receive locally generated answers with source citations.

This plan intentionally avoids specialized document domains. The initial system should support general formats such as TXT, Markdown, HTML, CSV, JSON, XML, PDF, DOCX, and XLSX where practical.

---

2. Core Architecture

Ashkorix should be built around a modular local pipeline:

```
IMPORT
file → detect type → extract text/data → normalize → chunk → hash → store
metadata

INDEX
chunks → embed → store vectors → index lexical text → preserve provenance

RETRIEVE
query → embed query → hybrid vector + keyword search → rank/fuse results

RERANK
candidate chunks → optional cross-encoder rerank → top context selection

GENERATE
build prompt → apply model chat template → run local GGUF model → stream
response

CITE
map generated [Source N] markers back to imported files, chunks, pages, or
sections
```

The system should be engine-first: build the backend and CLI harness before the desktop UI. The UI should consume the same stable application services rather than owning pipeline logic.

---

3. Proposed Workspace / Module Layout

A Rust workspace would fit the project well, but the same boundaries can apply in another language.

```
ashkorix/
├── crates/
│   ├── ashkorix-core/
│   │   ├── llm/                GGUF loading, chat templates, token streaming
│   │   ├── documents/          importers, extractors, normalization, chunking
│   │   ├── embeddings/         local embedding model support
│   │   ├── vectorstore/        local vector index or Qdrant wrapper
│   │   ├── search/             lexical index + hybrid fusion
│   │   ├── rerank/             optional local reranker
│   │   ├── rag/                retrieval orchestration
│   │   ├── cite/               citation assembly and marker mapping
│   │   ├── extensions/         plugin/extension host
│   │   └── config/             model paths, import settings, app config
│   ├── ashkorix-cli/           headless test harness
│   └── ashkorix-app/           desktop UI shell
├── extensions/
│   ├── importers/
│   ├── chunkers/
│   ├── metadata/
│   └── exporters/
└── fixtures/
    └── golden test corpora and expected outputs
```

---

4. Phase 0 — Project Skeleton and Local-Only Rules

Goal

Create the project foundation and enforce the local-only design philosophy from the start.

Build

Create the workspace, configuration system, logging, error handling, model directory conventions, and a CLI skeleton. Define the major service interfaces:

```
ModelService
DocumentImporter
Chunker
EmbeddingService
VectorIndex
LexicalIndex
RetrievalService
Reranker
PromptBuilder
CitationService
ExtensionHost
```

Local-only requirements

Ashkorix should never require cloud APIs for core functionality. The application should work with:

- local `.gguf` language models
- local embedding models
- local reranker models, if enabled
- local indexes and metadata databases
- local import extensions

Exit criteria

The CLI can start, read config, discover model files, list available importers, and run basic smoke tests.

---

5. Phase 1 — GGUF Model Runner

Goal

Build the core local model runner before adding document intelligence.

Build

Implement:

- `.gguf` model discovery
- model loading and unloading
- context size detection
- chat template detection
- prompt formatting
- token streaming
- stop sequence handling
- generation settings
- cancellation

- basic conversation history

Recommended generation controls

Expose simple settings first:

```
Temperature
Top-p
Top-k
Repeat penalty
Max tokens
Context size
GPU layers, if supported
Thread count
Seed
Stop sequences
```

Basic UI functionality for this phase

The first UI screen should be a simple local chat console:

- model selector
- load/unload button
- chat input
- streaming output
- generation settings drawer
- token speed display
- stop generation button
- copy response button
- clear conversation button

Exit criteria

A user can select a local `.gguf` model, load it, chat with it, stream responses, stop generation, and adjust basic inference settings.

---

6. Phase 2 — General File Import System

Goal

Import common file types into a normalized internal document format.

Initial supported formats

Start with:

```
TXT
Markdown
HTML
CSV
JSON
XML
PDF
DOCX
XLSX
```

The first milestone does not need advanced layout reconstruction. The goal is reliable text and table extraction, not perfect document rendering.

Internal document model

Each imported file should become a normalized record:

```
DocumentId
OriginalFilename
FilePath
FileType
ContentHash
ImportedAt
Title
Author, if available
CreatedDate, if available
ModifiedDate, if available
ExtractedText
ExtractedTables, if available
Metadata
```

Hashing and deduplication

Use file-byte hashing to detect duplicates. Store the full hash and a short display ID.

Exit criteria

The CLI and UI can import files, extract general content, store metadata, and avoid duplicate imports.

---

7. Phase 3 — Chunking and Metadata

Goal

Split imported documents into useful retrieval units.

Default chunking strategy

Use one general-purpose recursive chunker for v1:

```
Prefer headings
Then paragraphs
Then sentences
Then token/character windows
```

Each chunk should preserve:

```
ChunkId
DocumentId
CollectionId
Text
StartOffset
EndOffset
PageNumber, if available
SectionTitle, if available
Row/Sheet info, if available
SourceFilename
ContentHash
```

Optional chunker types for later

- semantic chunker
- table-aware chunker
- code-aware chunker
- conversation/log chunker
- heading hierarchy chunker

Basic UI functionality

Add an import review screen:

- list imported documents
- file type badges
- import status
- chunk count

- metadata preview
- extracted text preview
- delete/reimport document
- assign document to collection

Exit criteria

Files are imported, normalized, chunked, and available for indexing.

---

8. Phase 4 — Embeddings and Local Indexing

Goal

Create searchable local knowledge collections.

Build

Implement:

- local embedding model loading
- batch embedding generation
- vector storage
- lexical keyword indexing
- collection scoping
- index rebuild
- index health checks

Storage options

For a simple first version:

```
SQLite for metadata
tantivy or equivalent for lexical search
Qdrant or embedded vector index for vector search
filesystem storage for models and imported assets
```

For maximum portability, consider an embedded vector store later, but Qdrant is practical if the user is comfortable running a local service.

Basic UI functionality

Add a collections screen:

- create collection
- import files into collection

- show indexed/unindexed status
- rebuild index
- search collection
- view top matching chunks
- delete collection
- export collection metadata

Exit criteria

A user can import documents into a collection and search them locally using both semantic and keyword search.

---

9. Phase 5 — Hybrid Retrieval

Goal

Turn user questions into ranked context for the model.

Build

Implement:

- query embedding
- vector search
- lexical search
- result fusion
- collection filter
- duplicate chunk suppression
- context budget selection
- source ordering

Retrieval modes

Expose a few simple modes:

```
Fast
Balanced
Thorough
```

Suggested behavior:

- Fast: fewer chunks, no reranker
- Balanced: hybrid search with moderate chunk count
- Thorough: more candidates, reranker enabled if available

Basic UI functionality

Add a retrieval inspector:

- show retrieved sources before generation
- display chunk scores
- show whether result came from keyword, vector, or both
- allow user to exclude a source
- allow user to regenerate with different retrieval mode

Exit criteria

Questions return relevant, collection-scoped chunks with stable source ordering.

---

10. Phase 6 — Cited RAG Answers

Goal

Generate local answers grounded in imported sources.

Prompt structure

The RAG prompt should include:

```
System instruction
User question
Answer rules
Numbered source block
Citation requirement
Conversation context, if enabled
```

Example citation rule:

```
Use [Source N] markers when making claims based on the provided sources.
If the sources do not contain the answer, say so clearly.
Do not invent citations.
```

Citation model

Each source citation should include:

```
SourceNumber
DocumentId
```

```
OriginalFilename
PageNumber, if available
SectionTitle, if available
ChunkPreview
Score
CollectionName
```

Marker mapping

After generation, parse markers such as:

```
[Source 1]
[Source 2]
[Source 3]
```

Then map them back to the citation list. Mark unresolved citations as dangling instead of silently accepting them.

Basic UI functionality

Upgrade the chat screen:

- cited answer rendering
- clickable source chips
- source side panel
- source preview
- jump to document/chunk
- copy answer with citations
- copy source list
- warning when answer has no citations
- warning when the model cites a nonexistent source

Exit criteria

A user can ask a question over a collection and receive a locally generated answer with clickable citations.

---

11. Phase 7 — Import Extension System

Goal

Create a plugin-like system for adding new importers without modifying the core application.

Extension philosophy

Extensions should be capability-limited. They should not have unrestricted access to the filesystem, network, model runtime, or database.

The first extension category should be import extensions because they are easy to reason about:

```
Input file → extracted normalized document output
```

Recommended extension types

Start with four extension interfaces:

```
ImporterExtension
ChunkerExtension
MetadataExtractorExtension
ExporterExtension
```

Later extension types could include:

```
PromptTemplateExtension
PostProcessorExtension
SearchAdapterExtension
ModelAdapterExtension
```

Importer extension contract

An importer should declare:

```
ExtensionId
Name
Version
Author
SupportedExtensions
SupportedMimeTypes
RequiredPermissions
ConfigurationSchema
```

It should implement:

```
CanHandle(file)
Extract(file, options) → NormalizedDocument
ValidateOutput(document)
```

Extension packaging

Use a folder-based package:

```
extensions/
└── importers/
    └── example-importer/
        ├── extension.toml
        ├── plugin.wasm
        ├── README.md
        └── schema.json
```

Suggested extension manifest

```toml
id = "ashkorix.importers.example"
name = "Example Importer"
version = "0.1.0"
kind = "importer"
runtime = "wasm"

supported_extensions = ["example"]
supported_mime_types = ["application/example"]

[permissions]
read_input_file = true
write_temp_files = false
network = false

[entrypoints]
can_handle = "can_handle"
extract = "extract"
validate = "validate"
```

Recommended runtime

Use a WASM-based extension host if possible. WASM provides a safer boundary than loading arbitrary native libraries. Good candidates include:

```
Extism
Wasmtime
WASI component model
```

For a simpler early version, support built-in importers first and design the extension ABI before exposing third-party plugins.

Basic UI functionality

Add an extensions screen:

- list installed extensions
- enable/disable extension
- show supported file types
- show permissions
- configure extension options
- validate extension
- view extension logs
- uninstall extension

Exit criteria

Ashkorix can discover importer extensions, show them in the UI, and route matching files through either built-in or extension-provided importers.

---

12. Phase 8 — Desktop Application Shell

Goal

Build the main desktop app around the proven local engine.

Recommended screens

```
Chat
Models
Collections
Imports
Search
Sources
Extensions
Settings
Diagnostics
```

Chat screen

Core functionality:

- select model
- select collection
- ask question
- stream answer
- stop generation
- show citations
- open source panel
- regenerate
- change retrieval mode
- save conversation

Models screen

Core functionality:

- model folder selection
- list `.gguf` models
- show model metadata
- load/unload model
- configure runtime settings
- validate model file
- show RAM/VRAM estimate if available

Collections screen

Core functionality:

- create/delete collection
- add files
- show indexed status
- rebuild collection
- search collection
- inspect chunks

Imports screen

Core functionality:

- drag-and-drop files
- import progress
- failed import details
- reprocess file
- extension used
- extracted text preview

Extensions screen

Core functionality:

- manage import extensions
- enable/disable plugins
- configure plugin settings
- review plugin permissions
- view plugin logs

Settings screen

Core functionality:

- model directory
- data directory
- default collection
- default retrieval mode
- embedding model path
- reranker model path
- privacy/local-only status
- diagnostics export

Exit criteria

A nontechnical user can load a model, import files, ask questions, inspect citations, and manage extensions without using the CLI.

---

13. Phase 9 — Testing and Golden Fixtures

Goal

Make behavior stable and regression-resistant.

Fixture categories

Create deterministic tests for:

```
File hashing
Text extraction
Chunking
Embedding dimensions
Vector math
Lexical search
Hybrid fusion
```

```
Citation assembly
Prompt formatting
Token counting
Extension manifest parsing
Extension permission enforcement
```

Do not assert exact generated prose. Model output can vary. Instead, assert:

```
retrieved chunk IDs
source ordering
assembled prompt
citation list
resolved citation markers
```

Exit criteria

Core behavior is testable without relying on subjective inspection of model answers.

---

14. Phase 10 — Packaging and Distribution

Goal

Create a portable local application.

Package contents

A release package should include:

```
Ashkorix executable
Default config
Required native libraries
Optional local embedding model
Optional local reranker model
Extension directory
Model directory placeholder
User data directory setup
```

The application should not require users to install Python, Node, .NET, or a cloud service.

Packaging features

- first-run setup wizard
- model directory picker

- data directory picker
- local-only privacy notice
- diagnostics bundle exporter
- safe reset option
- index rebuild tool

Exit criteria

A user can download Ashkorix, point it at a `.gguf` model, import documents, and run local cited RAG without external services.

---

15. Suggested MVP Cut

The smallest useful Ashkorix MVP is:

```
Local GGUF chat
TXT / MD / HTML / CSV / JSON import
Basic PDF and DOCX import
Recursive chunking
Local embeddings
Hybrid search
Cited RAG answers
Collections
Simple desktop UI
Built-in importers
Extension manifest design
```

The plugin runtime itself can wait until after the MVP, but the importer architecture should be designed as if extensions will exist.

---

16. Later Enhancements

After the basic system works, consider:

```
WASM importer plugins
Semantic chunking
Table-aware chunking
Image OCR as optional local module
Local reranker
Conversation memory per collection
Source graph view
Document comparison
```

```
Saved prompt templates
Batch question runs
Export answer packets
Model benchmarking
GPU acceleration options
Multi-model routing
Local agent workflows
```

---

17. Development Priority Summary

Build in this order:

```
1. Project skeleton and local-only config
2. GGUF model runner
3. Basic chat UI
4. General file importers
5. Chunking and metadata
6. Embeddings and indexes
7. Hybrid retrieval
8. Cited RAG
9. Import extension architecture
10. Desktop app polish
11. Golden fixture tests
12. Packaging
```

The key design principle is to keep Ashkorix boring, local, testable, and extensible. First make the core loop reliable. Then make it pleasant. Then make it expandable.